

Reasoning with Knowledge

Kaiqi Zhao

Harbin Institute of Technology, Shenzhen



Outline

- Inference in Propositional Logic
- Inference in FOL

Recap: Propositional Knowledge Base

Question. What is a knowledge base?

- A set of clauses can be regarded as **knowledge** about the atomic propositions.
- For any proposition A , there is a set of clauses S such that any interpretation π satisfies A if and only if π satisfies S .

Definition

A **knowledge base** is a set of clauses.

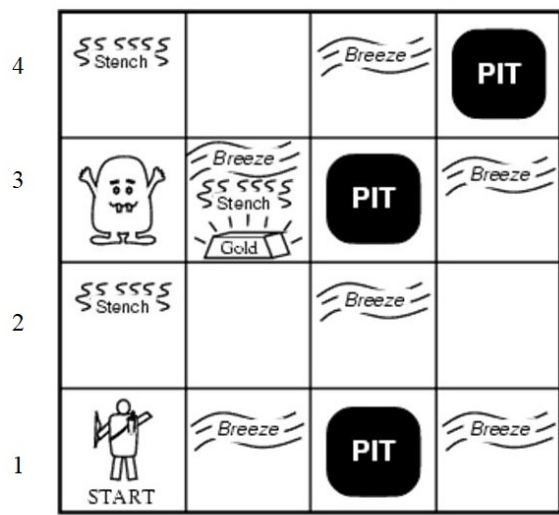
Definition

A **model** of a knowledge base KB is an interpretation that satisfies KB.

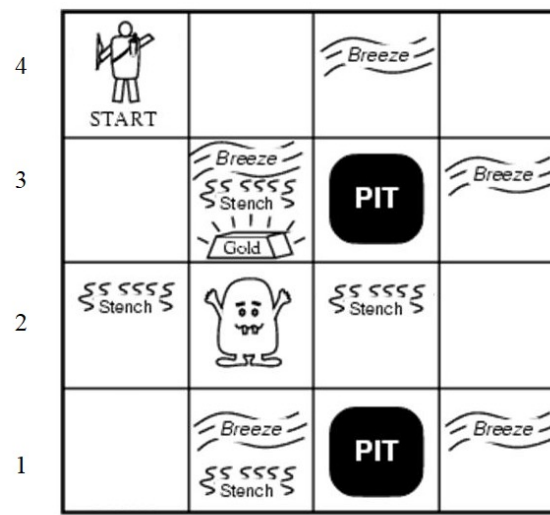
Example. [Wumpus world] KB contains

- $S_{i',j'} \leftarrow W_{i,j}$ and $B_{i',j'} \leftarrow P_{i,j}$ where $(i,j) \in \{1, \dots, 4\}^2$ and $|i' - i| + |j' - j| = 1$.
- $(W_{i-1,j} \vee W_{i,j-1} \vee W_{i+1,j} \vee W_{i,j+1}) \leftarrow S_{i,j}$ and $(P_{i-1,j} \vee P_{i,j-1} \vee P_{i+1,j} \vee P_{i,j+1}) \leftarrow B_{i,j}$ for $i,j \in \{2,3\}^2$.
- $(W_{2,1} \vee W_{1,2}) \leftarrow S_{1,1}$, $(P_{2,1} \vee P_{1,2}) \leftarrow B_{1,1}$, ... which describe if a location has wumpus or pit, then the surround locations have stench or breeze, respectively.

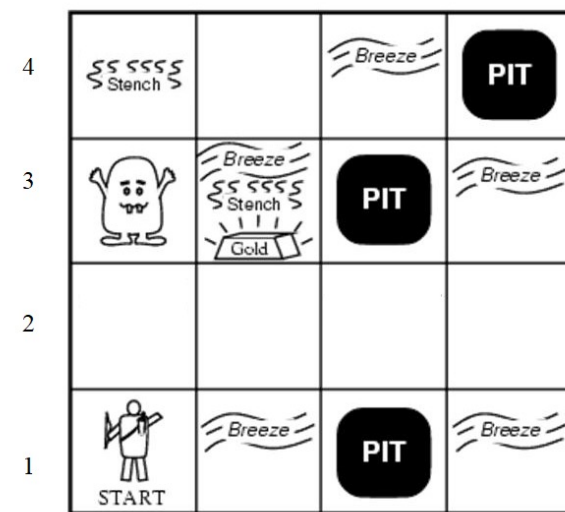
The following represents three interpretations:



A model of KB



A model of KB



Not a model

Propositional Inference Engine

A proposition g is called a **logical consequence** of a knowledge base KB, written as

$$\text{KB} \models g$$

if g is true in every model of KB.

Equivalently, we can say KB **entails** g

Example. In the last example, $W_{1,2} \rightarrow S_{1,3}$ is a logical consequence of KB. So

$$\text{KB} \models S_{1,3} \leftarrow W_{1,2}$$

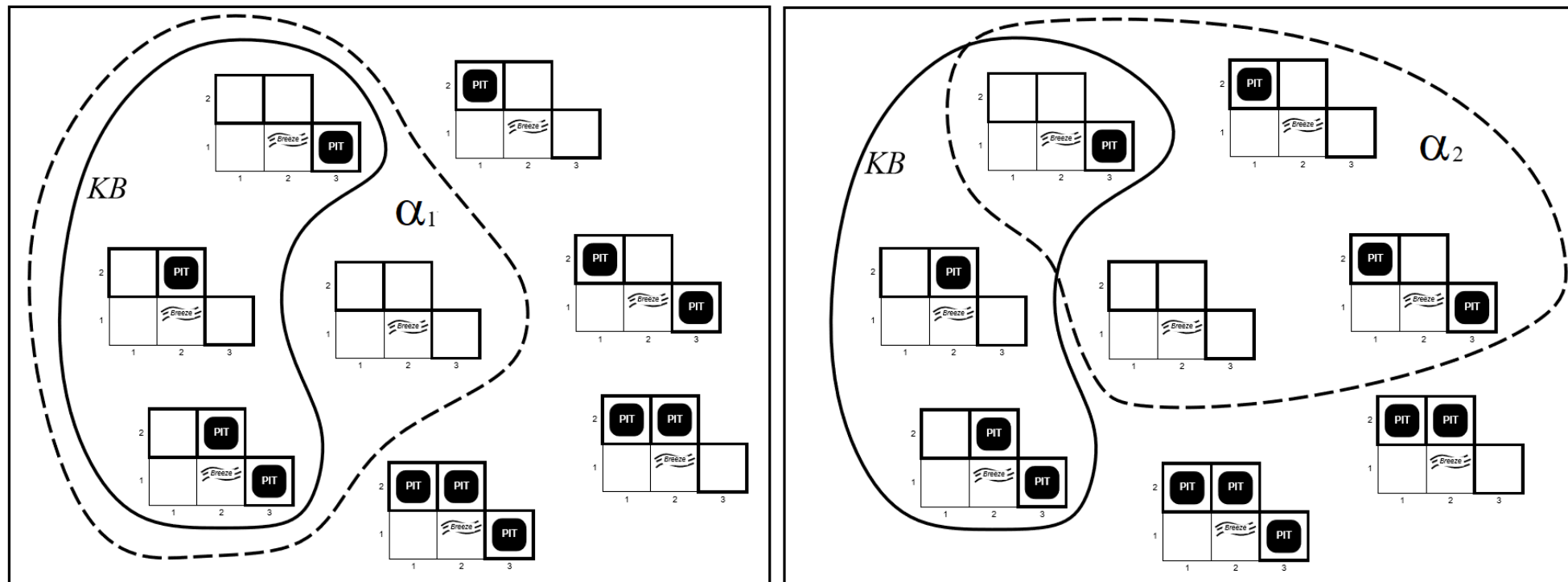
Similarly

$$\text{KB} \models \neg W_{2,3} \leftarrow \neg S_{2,2}$$

and

$$\text{KB} \models (W_{4,3} \vee W_{3,4}) \leftarrow (S_{3,3} \wedge \neg W_{2,3} \wedge \neg W_{3,2})$$

Example. [Wumpus world] Consider the KB for wumpus world and suppose the agent has visited cells (1,1) and (2,1) and observed $OB = \{\neg S_{2,1}, B_{2,1}, \neg S_{1,1}, \neg B_{1,1}, OK_{1,1}, OK_{2,1}\}$. Consider propositions $\alpha_1: \neg P_{1,2}, \alpha_2: \neg P_{2,2}$. Draw all interpretations that satisfy OB (only affecting cells (1,2), (2,2), (3,1)):



Note that $KB \cup OB \models \alpha_1$ but $KB \cup OB \not\models \alpha_2$

Example. Suppose KB contains the following propositions:

- $sam_is_happy \leftarrow night_time \wedge no_cloud,$
- $bird_eats_apple \leftarrow apple_is_red \wedge bird_around,$
- $bird_around \leftarrow no_cloud$
- $apple_is_eaten \leftarrow bird_eats_apple.$

Observation OB = {*apple_is_red*, *no_cloud*}

- 1) Are the following true?
 - $KB \cup OB \models bird_eats_apple$
 - $KB \cup OB \models apple_is_red$
 - $KB \cup OB \models sam_is_happy$
- 2) What are the models of $KB \cup OB$.
- 3) Give an interpretation that is not a model of $KB \cup OB$

Question. What is an inference engine?

Definition

An **inference engine** decides for any KB, a set of **observation atoms** OB, and proposition g , whether

$$KB \cup OB \models g$$

The proposition g is called a query.

Propositional Logic Inference

We summarize the problems of propositional logic inference as the entailment of a proposition (query), given a set of propositions (KB and OB).

Propositional Logic Inference Problem

INPUT: A set of propositions α , and proposition β

OUTPUT: Decide if $\alpha \models \beta$

Propositional Logic Inference v.s. Satisfaction Problem (SAT)

- A proposition is called **satisfiable** if there is an interpretation that satisfies the proposition.

E.g., $(p \vee \neg q) \wedge (\neg p \vee q)$ is satisfiable by $\pi(p) = \text{true}$ and $\pi(q) = \text{true}$.

$(p \vee \neg q) \wedge (\neg p \vee q) \wedge (\neg p \vee \neg q) \wedge (p \vee q)$ is not **satisfiable**.

- The **satisfaction problem (SAT)** is to ask for a satisfying interpretation of a proposition.

E.g., To solve $(p \wedge q \wedge \neg r) \vee (\neg q \rightarrow (\neg p \wedge r))$ is to find an interpretation π such that $(p \wedge q \wedge \neg r) \vee (\neg q \rightarrow (\neg p \wedge r))$ is true.

Theorem

For propositions α and β , $\alpha \models \beta$ if and only if $\alpha \wedge \neg\beta$ is not satisfiable.

Proof. Suppose $\alpha \models \beta$, but $\alpha \wedge \neg\beta$ is satisfiable.

- All models of α satisfies β
- Let π be the interpretation that satisfies $\alpha \wedge \neg\beta$, then $\pi(\neg\beta) = \text{true}$.
- Thus $\pi(\beta) = \text{false}$, contradiction.

Suppose $\alpha \not\models \beta$

- This means that there is a model of α that does not satisfy β .
- Call this model π .
- Then $\pi(\alpha \wedge \neg\beta) = \text{true}$.
- Thus if $\alpha \wedge \neg\beta$ is not satisfiable, we have $\alpha \models \beta$.

Propositional Logic Inference

Solving the propositional logic inference problem $\alpha \models \beta$ is **equivalent to** solving the SAT for $\alpha \wedge \neg\beta$.

However, SAT is a well-known **NP-hard problem**.

To simplify the problem, we will discuss two types of propositional KB models in which efficient inference can be performed:

- Definite Clause KB
- Horn Clause KB

Definite Clause

Definition

A **definite clause** is of the form $H \leftarrow (A_1 \wedge \cdots \wedge A_m)$
where $m \geq 0$, H and every A_i is an atom.

E.g.

- $S_{1,2} \leftarrow (W_{2,2} \wedge W_{1,3})$ and $A_{1,1}$ are definite clauses
- $W_{2,2} \leftarrow \neg S_{1,2}$ and $(S_{2,2} \wedge S_{1,3}) \leftarrow W_{1,2}$ are not definite clauses

A **definite clause knowledge base** is a knowledge base that contains only definite clauses.

The **definite clause inference engine** would need to handle queries of the form

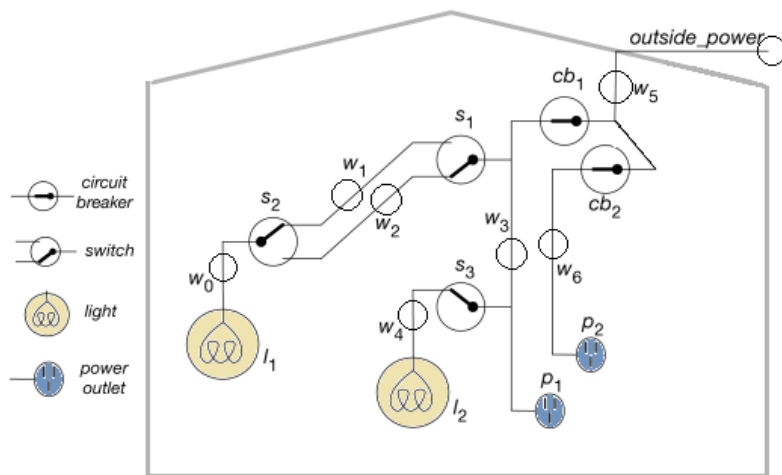
ask b

where b is an atom.

We often write $H \leftarrow (A_1 \wedge \cdots \wedge A_m)$ as $H \leftarrow A_1 \wedge \cdots \wedge A_m$.

Example. Consider the following electrical environment in a house.

- Atoms: $ok_l_1, ok_l_2, ok_cb_1, ok_cb_2, live_outside$, etc.
- KB is given below.



$live_l_1 \leftarrow live_w_0.$

$live_w_0 \leftarrow live_w_1 \wedge up_s_2.$

$live_w_0 \leftarrow live_w_2 \wedge down_s_2.$

$live_w_1 \leftarrow live_w_3 \wedge up_s_1.$

$live_w_2 \leftarrow live_w_3 \wedge down_s_1.$

$live_l_2 \leftarrow live_w_4.$

$live_w_4 \leftarrow live_w_3 \wedge up_s_3.$

$live_p_1 \leftarrow live_w_3.$

$live_w_3 \leftarrow live_w_5 \wedge ok_cb_1.$

$live_p_2 \leftarrow live_w_6.$

$live_w_6 \leftarrow live_w_5 \wedge ok_cb_2.$

$live_w_5 \leftarrow live_outside.$

$lit_l_1 \leftarrow live_l_1 \wedge ok_l_1.$

$lit_l_2 \leftarrow live_l_2 \wedge ok_l_2.$

Observations: $down_s_1, up_s_2, up_s_3, ok_cb_1, live_outside, ok_l_1, ok_l_2$.

The inference engine would return true or false for queries e.g.

ask lit_l_2 and **ask** lit_l_1

Question. How to implement a definite clause inference engine?

- An inference engine produces a **proof**, i.e., a mechanically derivable demonstration that a proposition g logically follows from KB.
- The algorithm that generates a proof is called a **proof procedure**. If there is a proof of g from KB, we write $\text{KB} \vdash g$.
- A proof procedure is **sound** if every proposition g that is derived from KB is a logical consequence, i.e., $\text{KB} \cup \text{OB} \vdash g$ implies $\text{KB} \cup \text{OB} \models g$.
- A proof procedure is **complete** if there is a proof of each logical consequence of the KB, i.e., $\text{KB} \cup \text{OB} \models g$ implies $\text{KB} \cup \text{OB} \vdash g$.

We next introduce two proof procedures that are both sound and complete.

1. Forward chaining
2. SLD resolution

Forward Chaining

- Start from clauses in $KB \cup OB$ and generate new logical consequences.
- Each derivation is built on the clauses in $KB \cup OB$ or the clauses that have already been generated.
- Use a **rule of derivation** for inference.

Modus Ponens

Modus ponens (MP) is the inference rule:

$$\frac{h \leftarrow a_1 \wedge \cdots \wedge a_m, \quad a_1, \dots, a_m}{h}$$

Forward chaining applies MP iteratively to the current knowledge to generate new clause, which are then added to knowledge.

Example. Suppose KB contains: (1) $a \leftarrow b \wedge c$, (2) $b \leftarrow d \wedge e$, (3) $b \leftarrow g \wedge e$, (4) $c \leftarrow e$, (5) $f \leftarrow a \wedge g$.

Observation OB contains (6) d , (7) e .

Suppose the query is **ask** a .

Forward chaining would develop the following proof:

(8) c	$MP(4), (7)$
(9) b	$MP(2), (6), (7)$
(10) a	$MP(1), (9), (8)$

Thus we can answer $KB \cup OB \vdash a$

ForwardChain(KB, OB, g)

INPUT: Definite clause knowledge base KB, Observation OB. Query of the form **ask** g .

OUTPUT: true if $\text{KB} \cup \text{OB} \vdash g$; false if $\text{KB} \cup \text{OB} \not\vdash g$.

create an empty set $C \leftarrow \emptyset$

repeat

 Select $h \leftarrow a_1 \wedge \cdots \wedge a_m$ in $\text{KB} \cup \text{OB}$ where $a_i \in C$ for all $1 \leq i \leq m, h \notin C$

if $h = g$ **then**

return true

end if

$C \leftarrow C \cup \{h\}$

Until C does not change any more

Return false

▷ Apply MP

Theorem [Soundness of Forward Chaining]

For any definite clause KB, OB and query g , $KB \cup OB \vdash g$ implies that $KB \cup OB \models g$.

Proof. We show that every atom a that is added to C by the algorithm is a **logical consequence** of $KB \cup OB$. **Proof by contradiction**

- Suppose there is an atom $h \in C$ that is not a logical consequence. Let h be **the first** ever such atom to be added in C .
- **By the algorithm**, to add h in C there must be some clause in $KB \cup OB$, in the form

$$h \leftarrow a_1 \wedge \cdots \wedge a_m$$

such that $a_1, \dots, a_m$ are all in C .

- **By assumption**, $KB \cup OB \models a_i$ for all $1 \leq i \leq m$.
- Then it must be that $KB \cup OB \models h$. **Contradiction.**

Theorem [Completeness of Forward Chaining]

For any KB, OB and query g , $KB \cup OB \models g$ implies that $KB \cup OB \vdash g$.

Idea of Proof.

- **Proof by contradiction.** Assume $KB \cup OB \not\models g$, prove it is incorrect.
- We can construct an interpretation I that all the atoms in C are true while others, **including g** , are false.
- Then, we only need to prove that I is a model of $KB \cup OB$. By the definition of entailment, **g must be true in every model of $KB \cup OB$, including I .** This results in a contradiction.

Theorem [Completeness of Forward Chaining]

For any KB, OB and query g , $KB \cup OB \models g$ implies that $KB \cup OB \vdash g$.

Proof. We only discuss the case when g is an atom. Suppose $KB \cup OB \not\models g$, and consider the resulting set C after running forward chaining.

- Define an **interpretation** I such that **for any atom a , $I(a) = \text{true}$ if and only if $a \in C$.**
- Since g is not in C , g is false in I .
- For any clause $h \leftarrow a_1 \wedge \cdots \wedge a_m$ in $KB \cup OB$. There are two situations:
 - $a_1, \dots, a_m$ are NOT all true in I . The clause is true by definition.
 - $a_1, \dots, a_m$ are all true in I . The algorithm will apply MP on $h \leftarrow a_1 \wedge \cdots \wedge a_m$ and adds h into C . Since $a_1, \dots, a_m$ and h are all true, the clause is true.
- Thus **I is a model of $KB \cup OB$.**
- Since $KB \cup OB \models g$, by definition, g must be true in every model of KB .
- In particular, g must be true in I . **Contradiction.**

Selective Linear Definite Clause (SLD) Resolution

- Start from the query g , and treat it as a **goal**.
- Represent the query as

$$yes \leftarrow g$$

where yes is a special atom that is always true.

- Infer **backwards**. Every step derives a clause

$$yes \leftarrow g_1 \wedge \cdots \wedge g_s$$

- Answer true if and only if $yes \leftarrow$ is derived.

Resolution

Resolution is the inference rule:

$$\frac{h \leftarrow a_1 \wedge \cdots \wedge a_m, a_m \leftarrow b_1 \wedge \cdots \wedge b_l}{h \leftarrow a_1 \wedge \cdots \wedge a_{m-1} \wedge b_1 \wedge \cdots \wedge b_l}$$

In the above, a_m is called a **subgoal**.

An **SLD derivation** of a query **ask** g from $KB \cup OB$ is a sequence of definite clauses $\gamma_0, \dots, \gamma_n$:

- The head of each γ_i is *yes*
- γ_0 is *yes* $\leftarrow g$
- For $i > 0$, γ_i is obtained by **resolution** from γ_{i-1} with a definite clause in $KB \cup OB$:

$$\frac{\gamma_{i-1}, \quad \text{a clause in } KB \cup OB}{\gamma_i}$$

- γ_n is *yes* $\leftarrow$

Example. Suppose KB contains: (1) $a \leftarrow b \wedge c$, (2) $b \leftarrow d \wedge e$, (3) $b \leftarrow g \wedge e$, (4) $c \leftarrow e$, (5) $f \leftarrow a \wedge g$.

Observation OB contains (6) d , (7) e .

Suppose the query is **ask** a .

A SLD derivation is

$yes \leftarrow a$	<i>Goal</i>
$yes \leftarrow b \wedge c$	<i>Res.(1)</i>
$yes \leftarrow c \wedge d \wedge e$	<i>Res.(2)</i>
$yes \leftarrow d \wedge e$	<i>Res.(4)</i>
$yes \leftarrow e$	<i>Res.(6)</i>
$yes \leftarrow$	<i>Res.(7)</i>

SLD Resolution(KB, OB, g)

INPUT: Definite clause knowledge base KB, query **ask** g .

OUTPUT: true if $\text{KB} \cup \text{OB} \vdash g$; false if $\text{KB} \cup \text{OB} \not\vdash g$.

Create a set $G \leftarrow \{g\}$

repeat

if KB does not contain a clause with head a for any $a \in G$ **then**

return false

end if

 Select an atom a in G

 Choose a definite clause $a \leftarrow b_1 \wedge \cdots \wedge b_m$ in $\text{KB} \cup \text{OB}$ with a as head

$G \leftarrow \{b_1, \dots, b_m\} \cup (G \setminus \{a\})$

until $G = \emptyset$

return true

Note. The algorithm described above may lead to a wrong path.

E.g. Suppose KB contains: ((1) $a \leftarrow b \wedge c$, (2) $b \leftarrow d \wedge e$, (3) $b \leftarrow g \wedge e$, (4) $c \leftarrow e$, (5) $f \leftarrow a \wedge g$. OB contains (6) d , (7) e . A possible execution is

$yes \leftarrow a$

$yes \leftarrow b \wedge c$

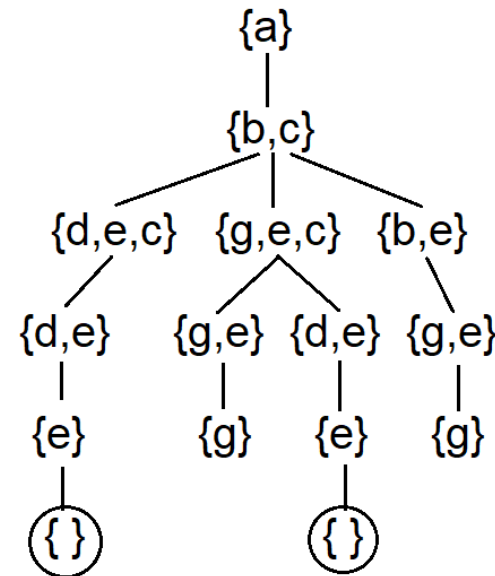
$yes \leftarrow g \wedge e \wedge c$

Goal

Res.(1)

Res.(3)

The SLD resolution implies a **search tree**:



- Thus SLD resolution can be performed using a [search algorithm](#), introduced in previous lectures.
- **Soundness of SLD Resolution:** If the search procedure has derived the goal, the rules used can be used by forward chaining to infer the query.
- **Completeness of SLD Resolution:** If forward chaining can derive an atom, then the rules used can be used to construct an SLD derivation¹.

¹For completeness of SLD resolution, we need to use a complete search method, e.g., BFS, ID, etc. that will not go into an infinite path

Horn Clauses

- Very often we would like to perform **diagnostic task**, i.e., derive the possible cause of an observation.
E.g. Given that the light l_2 is not lit, derive that some switches, wires, or circuit breakers are broken.
- A definite clauses KB cannot prove that a proposition is **false**.
- We need a stronger KB model.

Horn Clauses

- An **integrity constraint** is a clause of the form

$$no \leftarrow a_1 \wedge \cdots \wedge a_k$$

where each a_i is an atom and no is a special atom that is always false.

- A **Horn clause** is either a definite clause or an integrity constraint.

Definition

A Horn clause knowledge base is one that contains only Horn clauses.

Example. Consider the Horn clause KB containing

- $no \leftarrow a \wedge b$
- $a \leftarrow c$
- $b \leftarrow c$

In any model of KB, c must be false.

Therefore we have

$$KB \models \neg c$$

In a diagnostic task, we typically have the following:

- A knowledge base KB
- A set of atoms OB, i.e., **observations**.
- A set of atoms AS called the **assumables**. These are the potential causes for the observations.
- The inference uses **proof by contradiction**.

Definition

A **conflict** to a diagnostic task is a set of assumables $\{a_1, \dots, a_m\} \subseteq AS$ such that

$$KB \cup OB \models \neg a_1 \vee \dots \vee \neg a_m$$

Horn Clause Diagnosis

- INPUT: A knowledge base KB, observation set OB, assumable set AS.
- OUTPUT: A conflict $S \subseteq AS$ to the KB.

Question. How to solve the diagnosis problem?

Solve the diagnosis problem using an inference engine.

Example. Consider the KB containing

- $no \leftarrow a \wedge b$
- $a \leftarrow c$
- $b \leftarrow d$
- $b \leftarrow e$

Note that if we assume c, d are both true, KB derives no . In other words

$$KB \cup \{c, d\} \models no$$

Then we can conclude that

$$KB \models \neg c \vee \neg d$$

Thus the diagnosis problem can be seen as the problem of deriving *no*.

Example.

Consider the electrical environment:

live_outside.

live_l1 $\leftarrow$ *live_w0*.

live_w0 $\leftarrow$ *live_w1* $\wedge$ *up_s2* $\wedge$ *ok_s2*.

live_w0 $\leftarrow$ *live_w2* $\wedge$ *down_s2* $\wedge$ *ok_s2*.

live_w1 $\leftarrow$ *live_w3* $\wedge$ *up_s1* $\wedge$ *ok_s1*.

live_w2 $\leftarrow$ *live_w3* $\wedge$ *down_s1* $\wedge$ *ok_s1*.

live_l2 $\leftarrow$ *live_w4*.

live_w4 $\leftarrow$ *live_w3* $\wedge$ *up_s3* $\wedge$ *ok_s3*.

live_p1 $\leftarrow$ *live_w3*.

live_w3 $\leftarrow$ *live_w5* $\wedge$ *ok_cb1*.

live_p2 $\leftarrow$ *live_w6*.

live_w6 $\leftarrow$ *live_w5* $\wedge$ *ok_cb2*.

live_w5 $\leftarrow$ *live_outside*.

lit_l1 $\leftarrow$ *live_l1* $\wedge$ *ok_l1*.

lit_l2 $\leftarrow$ *live_l2* $\wedge$ *ok_l2*.

no $\leftarrow$ *dark_l1* $\wedge$ *lit_l1*.

no $\leftarrow$ *dark_l2* $\wedge$ *lit_l2*.

The diagnostic agent makes several observations:

$$OB = \{up_s_1, up_s_2, up_s_3, dark_l_1, dark_l_2\}$$

Note that

$$KB \cup OB \cup \{ok_cb_1, ok_s_1, ok_s_2, ok_l_1\} \models no$$

$$KB \cup OB \cup \{ok_cb_1, ok_s_3, ok_l_2\} \models no$$

Thus

$$KB \cup OB \models \neg ok_cb_1 \vee \neg ok_s_1 \vee \neg ok_s_2 \vee \neg ok_l_1 \text{ and}$$

$$KB \cup OB \models \neg ok_cb_1 \vee \neg ok_s_3 \vee \neg ok_l_2$$

Forward Chaining

- Maintain a set C that contains all atoms that can be derived from $KB \cup OB \cup AS$.
- In particular, every element in C is a pair (a, A) where a is an atom, and $A \subseteq AS$ such that

$$KB \cup OB \cup A \vdash a$$

- Initially, C contains $(a, \{a\})$ for each $a \in AS$.
- Iteratively expand C :
If $KB \cup OB$ contains $h \leftarrow b_1 \wedge \cdots \wedge b_m$ where each b_i has some $(b_i, A_i) \in C$, then add $(h, A_1 \cup \cdots \cup A_m)$ to C .

Example. Consider the KB containing

- $no \leftarrow a \wedge b$
- $a \leftarrow c \wedge d$
- $b \leftarrow d \wedge e$

OB = $\{d\}$ and AS = $\{c, e\}$.

Forward chaining may iteratively add the following pairs to C :

$$C = \{(c, \{c\}), (e, \{e\})\}$$

$$C = \{(c, \{c\}), (e, \{e\}), (d, \emptyset)\}$$

$$C = \{(c, \{c\}), (e, \{e\}), (d, \emptyset), (b, \{e\})\}$$

$$C = \{(c, \{c\}), (e, \{e\}), (d, \emptyset), (b, \{e\}), (a, \{c\})\}$$

$$C = \{(c, \{c\}), (e, \{e\}), (d, \emptyset), (b, \{e\}), (a, \{c\}), (no, \{c, e\})\}$$

Thus $KB \cup OB \cup \{c, e\} \vdash no$.

Conclusion: $KB \cup OB \vdash \neg c \vee \neg e$.

This forward chaining method is also called assumption-based truth maintenance system (ATMS).

ATMS(KB, OB, AS)

INPUT: Horn clause KB, Sets of atoms OB & AS.

OUTPUT: A set of conflicts

create $C \leftarrow \{(a, \{a\}) \mid a \in AS\}$

repeat

 Select clause $h \leftarrow b_1 \wedge \cdots \wedge b_m$ from $KB \cup OB$ such that

$(b_i, A_i) \in C$ for all $1 \leq i \leq m$ and $(h, A_1 \cup \cdots \cup A_m) \notin C$

$C \leftarrow C \cup (h, A_1 \cup \cdots \cup A_m)$

until No more selections are possible

return $\{A \mid (no, A) \in C\}$

SLD Resolution

- Apply the same SLD resolution steps as for definite clauses to generate a **SLD derivation**.
- The difference is that the goal is to derive *no*.

An **(diagnostic) SLD derivation** of a set of assumables *AS* from *KB* and *OB* is a sequence of integrity constraints $\gamma_0, \dots, \gamma_n$:

- The head of each γ_i is *no*.
- γ_0 is $\text{no} \leftarrow B$ that belongs to *KB*.
- For $i > 0$, γ_i is obtained by **resolution** from γ_{i-1} with a definite clause in $\text{KB} \cup \text{OB}$:
$$\frac{\gamma_{i-1}, \quad \text{a clause in KB}}{\gamma_i}$$
- γ_n is $\text{no} \leftarrow A$ where $A \subseteq \text{AS}$

This establishes that $\text{KB} \cup \text{OB} \cup A \vdash \text{no}$

Example. Consider the KB containing

- $no \leftarrow a \wedge b$
- $a \leftarrow c \wedge d$
- $b \leftarrow d \wedge e$

OB = $\{d\}$ and AS = $\{c, e\}$.

A possible SLD derivation:

- $no \leftarrow a \wedge b$
- $no \leftarrow c \wedge d \wedge b$
- $no \leftarrow c \wedge d \wedge e$
- $no \leftarrow c \wedge e$

Thus $KB \cup OB \cup \{c, e\} \vdash no$.

Conclusion: $KB \cup OB \vdash \neg c \vee \neg e$.

SLD Resolution(KB, g)

INPUT: Horn clause KB, Sets of atoms OB & AS.

OUTPUT: A conflict.

create a set $G \leftarrow \{no\}$

repeat

if KB does not contain a clause with head a for any $a \in G$ **then**

return false

end if

 Select an atom $a \in G$ such that $a \notin AS$

 Select a definite clause $a \leftarrow b_1 \wedge \cdots \wedge b_m$ in KB with a as head

$G \leftarrow \{b_1, \dots, b_m\} \cup (G \setminus \{a\})$

until $G \subseteq AS$

return G

Example. Consider the electrical environment example.

<i>live_outside.</i>	<i>live_p1</i> $\leftarrow$ <i>live_w3</i> .
<i>live_l1</i> $\leftarrow$ <i>live_w0</i> .	<i>live_w3</i> $\leftarrow$ <i>live_w5</i> $\wedge$ <i>ok_cb1</i> .
<i>live_w0</i> $\leftarrow$ <i>live_w1</i> $\wedge$ <i>up_s2</i> $\wedge$ <i>ok_s2</i> .	<i>live_p2</i> $\leftarrow$ <i>live_w6</i> .
<i>live_w0</i> $\leftarrow$ <i>live_w2</i> $\wedge$ <i>down_s2</i> $\wedge$ <i>ok_s2</i> .	<i>live_w6</i> $\leftarrow$ <i>live_w5</i> $\wedge$ <i>ok_cb2</i> .
<i>live_w1</i> $\leftarrow$ <i>live_w3</i> $\wedge$ <i>up_s1</i> $\wedge$ <i>ok_s1</i> .	<i>live_w5</i> $\leftarrow$ <i>live_outside</i> .
<i>live_w2</i> $\leftarrow$ <i>live_w3</i> $\wedge$ <i>down_s1</i> $\wedge$ <i>ok_s1</i> .	<i>lit_l1</i> $\leftarrow$ <i>live_l1</i> $\wedge$ <i>ok_l1</i> .
<i>live_l2</i> $\leftarrow$ <i>live_w4</i> .	<i>lit_l2</i> $\leftarrow$ <i>live_l2</i> $\wedge$ <i>ok_l2</i> .
<i>live_w4</i> $\leftarrow$ <i>live_w3</i> $\wedge$ <i>up_s3</i> $\wedge$ <i>ok_s3</i> .	<i>no</i> $\leftarrow$ <i>dark_l1</i> $\wedge$ <i>lit_l1</i> .
	<i>no</i> $\leftarrow$ <i>dark_l2</i> $\wedge$ <i>lit_l2</i> .

The diagnostic agent makes several observations:

$$OB = \{up_s_1, up_s_2, up_s_3, dark_l_1, dark_l_2\}$$

- Apply forward chaining to derive a conflict.
- Apply SLD resolution to derive a conflict.

Inference in FOL

First-order Inference Problem

Definition

- A sentence α is called a **logical consequence** of a knowledge base KB , written as $KB \models \alpha$ if α is true in every model of KB .
- The **inference problem** *Ask α* takes as input a sentence α and outputs whether $KB \models \alpha$

Example. Consider signature that contains unary relations: *King*, *Greedy*, *Evil* and binary relation *Brother*. The knowledge base KB contains:

$\forall x: King(x) \wedge Greedy(x) \rightarrow Evil(x)$

$King(John)$

$\forall y: Greedy(y)$

$Brother(Richard, John)$

The inference problem: *Ask Evil(John)* should return true

Basic tools

- **Grounding:**
 - Instantiate variables with constants
 - Propositionalise an FOL sentence to a proposition
 - Apply methods for propositional logic to perform inference
- **Lifting:**
 - Inference directly apply to first-order sentences
 - Generalise propositional methods to first-order logic
 - Unification is an important task

Method 1: Grounding

- Our aim is to convert the FOL inference problem to propositional logic inference
- Since we have variables in FOL, we need to **instantiate** these variables.
- **Substitution:**
 - A substitution is of the form $\theta = \{x/t\}$ where x is a variable and t is a constant.
 - By convention, the substitution operation on a sentence α is denoted by $\text{Subst}(\theta, \alpha)$, or $\alpha\theta$.
 - E.g., $\text{Subst}(\{x/c\}, R(x, 5)) = R(c, 5)$
- **Quantifiers:**
 - A sentence without any quantifier is a proposition, e.g., *King(John)* and *Greedy(John)*.
 - However, when a sentence include universal quantifiers $\forall x: \varphi(x)$ and existential quantifiers $\exists x: \varphi(x)$, we need to convert the sentence into propositions.

Existential instantiation (EI)

For any sentence α , variable v , and constant symbol k that does **not** appear elsewhere in the knowledge base:

$$\frac{\exists v \alpha}{\text{Subst}(\{v/k\}, \alpha)}$$

E.g., $\exists x \text{Crown}(x) \wedge \text{OnHead}(x, \text{John})$ yields:
 $\text{Crown}(C_1) \wedge \text{OnHead}(C_1, \text{John})$

provided C_1 is a new constant symbol, called **a Skolem constant** — name must not already be in use

Universal instantiation (UI)

Every instantiation of a universally quantified sentence is entailed by it:

$$\frac{\forall v \alpha}{\text{Subst}(\{v/g\}, \alpha)}$$

for any variable v and ground term g

E.g., $\forall x \text{King}(x) \wedge \text{Greedy}(x) \rightarrow \text{Evil}(x)$ yields:

$\text{King}(\text{John}) \wedge \text{Greedy}(\text{John}) \rightarrow \text{Evil}(\text{John})$

$\text{King}(\text{Richard}) \wedge \text{Greedy}(\text{Richard}) \rightarrow \text{Evil}(\text{Richard})$

$\text{King}(\text{Father}(\text{John})) \wedge \text{Greedy}(\text{Father}(\text{John})) \rightarrow \text{Evil}(\text{Father}(\text{John}))$

...

Reduction to propositional inference

Suppose the KB contains just the following:

$\forall x \text{ King}(x) \wedge \text{Greedy}(x) \rightarrow \text{Evil}(x)$

$\text{King}(\text{John})$

$\text{Greedy}(\text{John})$

$\text{Brother}(\text{Richard}, \text{John})$

Instantiating the universal sentence in **all possible** ways, we have:

$\text{King}(\text{John}) \wedge \text{Greedy}(\text{John}) \rightarrow \text{Evil}(\text{John})$

$\text{King}(\text{Richard}) \wedge \text{Greedy}(\text{Richard}) \rightarrow \text{Evil}(\text{Richard})$

$\text{King}(\text{John})$

$\text{Greedy}(\text{John})$

$\text{Brother}(\text{Richard}, \text{John})$

The new KB is **propositionalized**: proposition symbols are

$\text{King}(\text{John}), \text{Greedy}(\text{John}), \text{Evil}(\text{John}), \text{King}(\text{Richard}), \text{etc.}$

Reduction contd.

Every FOL KB can be propositionalized so as to **preserve entailment**
(A ground sentence is entailed by new KB iff entailed by original KB)

Idea: propositionalize KB and query, apply resolution, return result

Problem: with function symbols, there are infinitely many ground terms,
e.g., $Father(Father(Father(John)))$

Reduction contd.

Theorem: Herbrand (1930). If a sentence α is entailed by an FOL KB, it is entailed by a **finite** subset of the propositionalized KB

Idea: For $n = 0$ to ∞ do

create a propositional KB by instantiating with depth- n terms see if α is entailed by this KB

Problem: works if α is entailed, loops if α is not entailed

Theorem: Turing, Church (1936). Entailment for FOL is **semidecidable** (algorithms exist that say **yes to every entailed sentence**, but no algorithm exists that also says **no to every nonentailed sentence**.)

Problems with Propositionalization

Propositionalization seems to generate lots of irrelevant sentences.

E.g., from: $\forall x \text{ King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x)$
 $\text{King}(\text{John})$
 $\forall y \text{ Greedy}(y)$
 $\text{Brother}(\text{Richard}, \text{John})$

It seems obvious that $\text{Evil}(\text{John})$, but propositionalization produces lots of facts such as $\text{Greedy}(\text{Richard})$ that are irrelevant

With p k -ary predicates and n constants, there are $p \cdot n^k$ instantiations.

Method 2: Lifting

We can get the inference immediately if we can find a substitution θ such that $King(x) \wedge Greedy(x)$ match $King(John) \wedge Greedy(y)$

Unification: A unification algorithm takes 2 sentences α and β and returns a substitution such that $Subst(\theta, \alpha) = Subst(\theta, \beta)$, written as:

$$Unify(\alpha, \beta) = \theta$$

E.g., $\theta = \{x/John, y/John\}$ works for the above two sentences

p	q	θ
$Knows(John, x)$	$Knows(John, Jane)$	$\{x/Jane\}$
$Knows(John, x)$	$Knows(y, OJ)$	$\{x/OJ, y/John\}$
$Knows(John, x)$	$Knows(y, Mother(y))$	$\{y/John, x/Mother(John)\}$
$Knows(John, x)$	$Knows(x, OJ)$	$fail$

Unification: A unification algorithm takes 2 sentences α and β and returns a substitution such that $Subst(\theta, \alpha) = Subst(\theta, \beta)$, written as:

$$Unify(\alpha, \beta) = \theta$$

The substitution θ that unifies two sentences is called a **unifier**.

Note. There can be multiple unifiers of two sentences

Example. To unify $Knows(John, x)$ and $Knows(y, z)$,
 $\theta_1 = \{y/John, x/z\}$ or $\theta_2 = \{y/John, x/John, z/John\}$

Definition

Unifier θ_1 is **more general** than unifier θ_2 if θ_2 can be obtained by adding substitutions to θ_1 .

Example. $\theta_1 = \{y/John, x/z\}$ is more general than $\theta_2 = \{y/John, x/John, z/John\}$

There is a single **most general unifier (MGU)** that is unique up to renaming of variables.

Standardize apart

Variable names may clash

p	q	θ
$Knows(John, x)$	$Knows(John, Jane)$	$\{x/Jane\}$
$Knows(John, x)$	$Knows(y, OJ)$	$\{x/OJ, y/John\}$
$Knows(John, x)$	$Knows(y, Mother(y))$	$\{y/John, x/Mother(John)\}$
$Knows(John, x)$	$Knows(x, OJ)$	$fail$

Standardizing apart (renaming its variables to avoid name clashes) eliminates overlap of variables, e.g., $Knows(z_{17}, OJ)$

The Unification Algorithm

- Recursively explore two expressions side by side, building a unifier along the way
- Declare failure if the two corresponding points in the structures do not match
- **Note.** When matching a **variable** against a **complex term**, we must check if the variable itself occurs inside the term.

Example.

$$\begin{aligned} & \text{Unify}(\text{Knows}(\text{John}, \text{mother}(x)), \text{Knows}(y, \text{mother}(\text{Jane})), \emptyset) \\ &= \text{Unify}([\text{John}, \text{mother}(x)], [y, \text{mother}(\text{Jane})], \emptyset) \\ &= \text{Unify}(\text{mother}(x), \text{mother}(\text{Jane}), \text{Unify}(\text{John}, y, \emptyset)) \\ &= \text{Unify}(\text{mother}(x), \text{mother}(\text{Jane}), \{y/\text{John}\}) \\ &= \text{Unify}(x, \text{Jane}, \{y/\text{John}\}) \\ &= \{y/\text{John}, x/\text{Jane}\} \end{aligned}$$

The Unification Algorithm

Example.

$$\begin{aligned} & \text{Unify}(\text{Knows}(\text{John}, x), \text{Knows}(y, \text{mother}(x)), \emptyset) \\ &= \text{Unify}([\text{John}, x], [y, \text{mother}(x)], \emptyset) \\ &= \text{Unify}(x, \text{mother}(x), \text{Unify}(\text{John}, y, \emptyset)) \\ &= \text{Unify}(x, \text{mother}(x), \{y/\text{John}\}) \\ &= \text{fail} \end{aligned}$$

Unify (x, y, θ)

INPUT: x, y variables, constants, lists or compound expressions, θ the substitution built thus far

OUTPUT: mgu θ' such that $\text{Subst}(\theta', x) = \text{Subst}(\theta', y)$

```
if  $\theta = \text{failure}$  then
    return failure
else if  $x = y$  then
    return  $\theta$ 
else if IsVariable( $x$ ) then
    return UnifyVar( $x, y, \theta$ )
else if IsVariable( $y$ ) then
    return UnifyVar( $y, x, \theta$ )
else if IsCompound( $x$ ) or IsCompound( $y$ ) then
    return Unify( $x.\text{args}, y.\text{args}, \text{Unify}(x.\text{op}, y.\text{op}, \theta)$ )
else if IsList( $x$ ) or IsList( $y$ ) then
    return Unify( $x.\text{rest}, y.\text{rest}, \text{Unify}(x.\text{first}, y.\text{first}, \theta)$ )
else
    return failure
end if
```

UnifyVar (x, y, θ)

INPUT: x variable, y variable, constant, list or compound expression, θ the substitution built thus far

OUTPUT: mgu θ' such that $\text{Subst}(\theta', x) = \text{Subst}(\theta', y)$

if $\{x/v\} \in \theta$ for some v **then**

return $\text{Unify}(v, y, \theta)$

else if $\{y/v\} \in \theta$ for some v **then**

return $\text{Unify}(x, v, \theta)$

else if $\text{OccurCheck}(x, y)$ **then**

return *failure*

else

return $\theta \cup \{x/y\}$

end if

▷ Check if variable x appears in y

Generalized Modus Ponens (GMP)

$$\frac{p'_1, p'_2, \dots, p'_n, (p_1 \wedge p_2 \wedge \dots \wedge p_n \rightarrow q)}{\text{Subst}(\theta, q)} \quad \text{where } \text{Subst}(\theta, p_i') = \text{Subst}(\theta, p_i) \text{ for all } i$$

p'_1 is *King(John)* p_1 is *King(x)*
 p'_2 is *Greedy(y)* p_2 is *Greedy(x)*
 θ is $\{x/\text{John}, y/\text{John}\}$ q is *Evil(x)*
 $\text{Subst}(\theta, q)$ is *Evil(John)*

GMP used with KB of **definite clauses**

All variables assumed universally quantified

Example knowledge base

The law says that it is a crime for a New Zealander to sell alcohol to minors. The girl Lucy is 15 years old and has some beers. All of the beers were sold to her by David, who is a New Zealander.

Prove that David is guilty.

Example knowledge base contd.

... it is a crime for a New Zealander to sell alcohol to minors:

$$NZ(x) \wedge Alcohol(y) \wedge Sells(x,y,z) \wedge Minor(z) \rightarrow Crime(x)$$

Lucy is 15 years old and has some beers:

$$Owns(Lucy,B) \text{ and } Beer(B) \text{ and } Under17(Lucy)$$

... all of the beers were sold to her by David

$$Beers(x) \wedge Owns(Lucy,x) \rightarrow Sells(David,x,Lucy)$$

Beers are alcohol:

$$Beers(x) \rightarrow Alcohol(x)$$

A minor is a person under 17 years old:

$$Under17(x) \rightarrow Minor(x)$$

David, who is a New Zealander ...

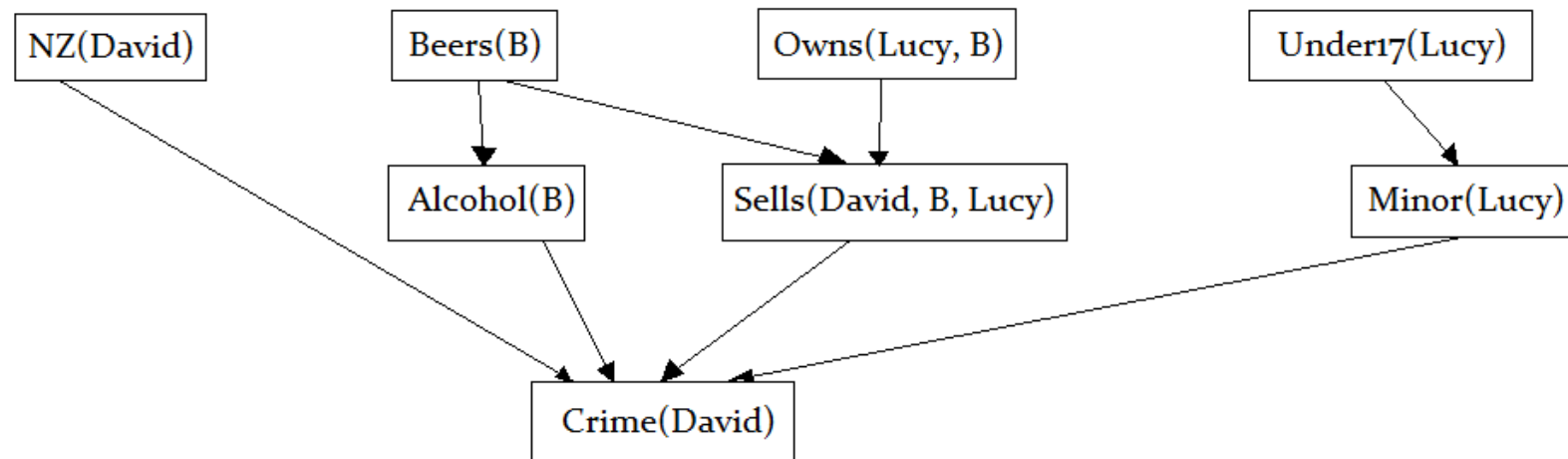
$$NZ(David)$$

Query: *Ask Crime(David)*

The Forward Chaining Algorithm

Idea: For a **definite clause** KB,

- Start with atom sentences in the KB
- Apply GMP and add new atomic sentences produced by GMP.
- Termine when no further inference can be made or the goal is reached



The Forward Chaining Algorithm

```
function FOL-FC-Ask( $KB, \alpha$ ) returns a substitution or false
  repeat until new is empty
     $new \leftarrow \{ \}$ 
    for each sentence  $r$  in  $KB$  do
       $(p_1 \wedge \dots \wedge p_n \Rightarrow q) \leftarrow \text{STANDARDIZE-APART}(r)$ 
      for each  $\theta$  such that  $(p_1 \wedge \dots \wedge p_n)\theta = (p'_1 \wedge \dots \wedge p'_n)\theta$ 
        for some  $p'_1, \dots, p'_n$  in  $KB$ 
           $q' \leftarrow \text{SUBST}(\theta, q)$ 
          if  $q'$  is not a renaming of a sentence already in  $KB$  or new then do
            add  $q'$  to new
             $\phi \leftarrow \text{UNIFY}(q', \alpha)$ 
            if  $\phi$  is not fail then return  $\phi$ 
    add new to  $KB$ 
  return false
```

Efficiency of Forward Chaining

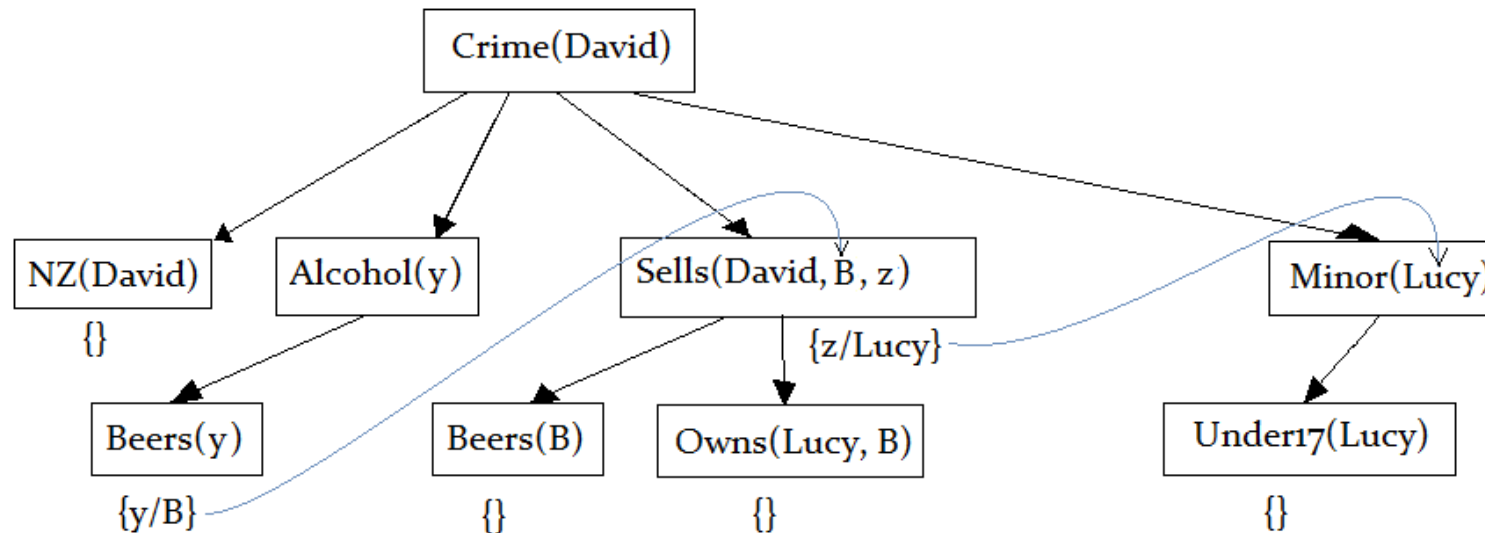
The basic forward chaining algorithm can be **inefficient** for three reasons:

- **Conjunct ordering**: The time to find a unifier θ depends on the ordering of $p_1, \dots, p_n$. E.g., $Beers(x) \wedge Owns(Lucy, x) \rightarrow Sells(David, x, Lucy)$. Many x can satisfy $Beers(x)$, but not many satisfies $Owns(Lucy, x)$
- **Redundant checks**: It checks over every rule at every iteration. E.g., Once $Alcohol(B)$ is added to KB, the rule $Alcohol(x) \leftarrow Beers(x)$ will be checked and $Alcohol(x)$ will be attempted again
- **Irrelevant facts**: It may generate many irrelevant facts. E.g., If KB contains $Beers(C)$, then $Alcohol(C)$ will be generated, but it is irrelevant to the query.

The Backward Chaining Algorithm

Idea: For a **definite clause** KB,

- Start with query (or the “goal”) and work backward
- Attempt to “unresolve” the goal into a set of pre-existing facts.



Backward chaining algorithm

```
function FOL-BC-Ask(KB, goals,  $\theta$ ) returns a set of substitutions
  inputs: KB, a knowledge base
           goals, a list of conjuncts forming a query ( $\theta$  already applied)
            $\theta$ , the current substitution, initially the empty substitution  $\{ \}$ 
  local variables: answers, a set of substitutions, initially empty

  if goals is empty then return  $\{ \theta \}$ 
   $q' \leftarrow \text{SUBST}(\theta, \text{FIRST}(\textit{goals}))$ 
  for each sentence r in KB
    where STANDARDIZE-APART(r) = ( $p_1 \wedge \dots \wedge p_n \Rightarrow q$ )
    and  $\theta' \leftarrow \text{UNIFY}(q, q')$  succeeds
     $\textit{new\_goals} \leftarrow [p_1, \dots, p_n | \text{REST}(\textit{goals})]$ 
     $\textit{answers} \leftarrow \text{FOL-BC-ASK}(\textit{KB}, \textit{new\_goals}, \text{COMPOSE}(\theta', \theta)) \cup \textit{answers}$ 
  return answers
```

Summary of The Topic

- Propositional inference problem
 - Definite clause
 - Forward chaining algorithm for definite clause reasoning.
 - SLD resolution for definite clause reasoning.
 - Horn clause
 - Describe forward chaining algorithm for Horn clause diagnosis.
 - Describe SLD resolution for Horn clause diagnosis.
- FOL inference problem
 - Reducing first-order inference to propositional inference
 - Unification
 - Generalized Modus Ponens
 - Forward chaining
 - Backward chaining